What to practise from this final

Working scope: Lectures 1-9. These are the decks in your Midterm folder; your existing revision guide also records Lectures 1-9 as confirmed coverage. I did not find a separate official midterm notice. This is a content comparison, not a prediction of this year's questions.

22 clear matches

Questions	Why they fit / lecture evidence
1, 11, 24	Variables, expressions and identifiers - L1, pp.25-51.
2, 20, 22	Functions, const-reference arguments, overloading - L2 pp.3-9,16-30; L8 p.22.
3, 4	String concatenation - L4 pp.24-26; stream operator/reference return - L8 pp.36-39.
5, 6, 7, 14, 15, 26, 27, 31, 32	Array size, pointers, addressing and expression tracing - L6 pp.9-11,18-29,35-46. Q26 mentions linked structures only as a use of pointers.
25	Static storage, stack and heap - L6 p.14; L7 p.4.
9	begin()/end() iterator meaning - L9 pp.16-22.
12, 21, 30	Encapsulation, OOP and virtual/nonvirtual dispatch - L8 pp.15-18,41-47,54-57.

Three optional questions

Q10: C++ paradigms; functional programming is not developed in these decks. **Q17:** bitwise shifts and masks; the operators appear in L1 p.42 and L6 p.32, but pixel masking is not a developed example. **Q23:** range-for is covered, but the exact Stanford container support is library-specific. Keep these for a second pass.

Seven questions to set aside for now

Q8 hashing and collision complexity; **Q13** merge-sort complexity; **Q16** minimax; **Q18** BST traversals; **Q19** heap priority queues; **Q28** recursive problem decomposition; **Q29** circular queue implementation. These are not developed in Lectures 1-9. L1 p.13 lists data structures and algorithms as later course parts. Selection sort as a template example and CharStack as a class-design example **are** still in scope.

What this paper leaves out

“Missing” means not directly assessed in this paper; a passing mention is not meaningful practice. Do not use this final as your complete midterm checklist.

Lecture	Coverage gap to revise
L1 | Foundations	Scope/lifetime, namespace and compilation/linking, control-flow tracing, short-circuit evaluation, casts, assertions. Q1/11/24 cover only a small slice.
L2 | Functions	Default parameters, pass-by-value vs reference mutation, writing/tracing swap, prototypes and algorithm implementation. Q2/20/22 are conceptual.
L3 | Libraries	Headers vs implementation, include guards, exported types/constants, enum operators, library design, rand/srand and seed behavior. Almost no direct coverage.
L4 | Strings	C-string terminators and storage, strlen/strcpy/strcmp, character classification, indexing, substr/find/erase/insert/replace and string-processing loops. Q3 tests only concatenation.
L5 | Streams	setw, fill/alignment, fixed/scientific and precision; whitespace handling; get/getline; file open/read/write/close; success/failure-controlled loops; stringstream conversion. Q4 and cout in Q31 do not cover these skills.
L6 | Pointers / arrays	Pointer-based swap, object pointers and ->/this, function pointers, writing C-string traversal/copy routines. Basic addressing is well represented.
L7 | Dynamic memory	new/delete and new[]/delete[], dynamic 2D arrays, leaks/dangling pointers, lifetime/destructors, malloc/free, const/static variations, unique/shared/weak pointers. Q25/26 only skim the concepts.
L8 | Classes	Constructors/initializer lists/destructors, shallow vs deep copy, copy constructor vs assignment, implementing operators, abstract classes/pure virtual functions, CharStack ownership, Rational/TokenScanner examples. Q30 is only one dispatch trace.
L9 | Templates / iterators	Writing function/class templates, multiple type parameters and return conversion, template definitions in headers, generic selection sort, forward/reverse iterator loops and implementation. Q9 tests only endpoint meaning.

Highest-priority gaps: L5 streams, L7 ownership/lifetime, L8 copying, and L9 templates. The final contains no substantial template-writing or file/string-stream exercise.

Pointers and arrays | Start here

Practise Q6, 7, 15 and 27, then Q31 and Q32. Draw a box for each variable and an arrow for each pointer. After every statement, update both the pointer position and the array contents. L6 pp.22-29 and 35-46 are the closest lecture examples.

Q31 worked trace - cover this until you try it

Statement	Value assigned	Pointer / array after statement
int* p = arr	-	p -> arr[0]; arr = {2,0,2,1}
int a = *++p	a = 0	p -> arr[1]; array unchanged
int b = *p++	b = 0	p -> arr[2]; array unchanged
int c = ++*p	c = 3	p -> arr[2]; arr = {2,0,3,1}
int d = (*p)++	d = 3	p -> arr[2]; arr = {2,0,4,1}

```
2041
0033
```

***p++** is ***(p++)**: use the old pointed-to value, then advance p. ***++p** advances p before dereferencing. **++*p** and **(*p)++** change the element, not p.

Q32: type determines the step size

In most expressions doubleArray becomes a double*; adding 1 advances 8 bytes. &doubleArray has type double (*)[10]; adding 1 advances 80 bytes (0x50). Same starting address, different types.

Expression(s)	Requested hexadecimal value
doubleArray; &doubleArray; &doubleArray[0]	ACE0 (addresses)
*doubleArray; doubleArray[0]	0000 (numeric value 0)
doubleArray+1; &doubleArray[1]	ACE8 (address)
&doubleArray+1	AD30 (one past the whole array)
doubleArray+9; &doubleArray[9]	AD28 (address: ACE0 + 9 x 8)
*doubleArray+9	0009 (numeric value: 0 + 9)
*(doubleArray+9); doubleArray[9]	0022 (numeric value: decimal 34)

The paper asks you to express numeric values in hex; this does not ask for the IEEE-754 bit representation of a double. One-past pointers may be formed, but not dereferenced.

Streams | A manageable checklist

Streams deserve deliberate practice because this final barely tests them. Work through L5 pp.11-17 (formatting/input), pp.20-31 (files/reading), and pp.34-38 (string streams).

1. Know what is consumed

>> normally skips leading whitespace; extraction into a string stops at whitespace. **get(ch)** reads the next character, including whitespace. **getline** reads through the delimiter and discards it. After number extraction, a newline can remain for the next getline. **ws** consumes leading whitespace; use it only when leading whitespace and blank lines should be ignored.

2. Track formatting state

setw(n) sets a minimum field width for the next formatted value. **setfill**, alignment, fixed/scientific and precision persist until changed. With **fixed**, precision counts decimal places; in the default format, it counts significant digits. A width never truncates a value.

3. Let the read operation control the loop

```
while (in >> value) { /* process value */ }
while (getline(in, line)) { /* process line */ }
while (in.get(ch)) { /* process character */ }
```

Check that opening a file succeeded. Do not use while (!in.eof()) as a general read loop: EOF is detected by a read attempt. L5 p.24 shows that pattern; p.30 provides the safer getline-controlled loop. clear() resets error flags; it does not itself remove invalid input.

4. Treat a string as an input/output source

Use istringstream to parse existing text; ostringstream to build formatted text; str() to obtain the stored string. Be able to trace a failed conversion and explain when a loop stops.

5. Try these three short drills

Assume the standard headers and using namespace std. Answers are on page 6. These are new practice exercises, not questions from the final.

```
// A: write the exact output
cout << fixed << setprecision(2) << setfill('_')
     << setw(6) << 3.5 << '|' << 2.0;

// B: what do s and t contain?
istringstream in("42\nAlice Bob\n");
int n; string s, t;
in >> n; getline(in, s); getline(in, t);

// C: what are total and in.fail() at the end?
istringstream in("10 20 x 30");
int x, total = 0;
while (in >> x) total += x;
```

Your revision priorities

These priorities combine the lecture coverage, this paper's practice value, and your concerns. They are study recommendations, not estimated exam probabilities.

Priority	What you should be able to do without notes
1 | Pointers + arrays	Trace Q31/32 accurately; distinguish p, *p, &p, arr and &arr; scale pointer arithmetic; explain array decay and bounds.
2 | Streams	Trace whitespace and newlines; predict exact padding/precision; write read-controlled loops; parse a string stream and detect failure.
3 | Dynamic memory + copying	Pair allocation/deallocation; draw stack/heap lifetime; identify leaks/dangling pointers; explain deep copy and object ownership.
4 | Classes + templates	Trace constructor/copy/assignment behavior and virtual dispatch; implement basic operators; write a small generic function/class and iterator loop.
5 | Strings + foundations	Practise C-string vs std::string operations, references/default arguments, integer division/control flow, library interfaces and random seeds.

A focused four-hour practice session

Time	Task
0:00-0:55	Attempt Q6/7/15/27/31/32. Check the trace on page 3 only after finishing. Redo errors with a memory diagram.
0:55-1:45	Do the stream drills; then write one line-reading loop and one numeric string-stream parser. Revisit L5 examples.
1:45-2:25	Practise new[]/delete[], a dynamic matrix cleanup, const pointer forms, and a deep-copy class.
2:25-3:00	Do Q12/20/21/22/30; trace a virtual vs nonvirtual call and distinguish copy construction from assignment.
3:00-3:30	Write a function template, trace a mixed-type return, and practise begin/end and reverse traversal.
3:30-4:00	Review strings, default arguments and library gaps; redo your three most frequent errors without notes.

Readiness check

You are ready to move on when you can explain **why** each pointer moves, which input characters remain unread, and who owns each allocated object. For every snippet, first decide: valid execution, compile error, or undefined behavior.

A note about old-paper wording

Q4 has misleading wording: std::ostream is noncopyable, so returning a normal output stream by value does not simply produce a copy. Learn ostream& and chaining. Q21 uses a broad “distinguishing features” formulation; abstraction also appears in other paradigms. Treat these as concept prompts rather than rely on an unverified old answer key.

Drill answers and source map

Streams: check your reasoning

A: `__3.50|2.00`. The first number occupies four characters, so width 6 adds two underscores. setw applies to that value; fixed and precision remain for the second number.

B: n = 42, s is empty, and t = "Alice Bob". The number extraction leaves the newline. The first getline consumes it and returns an empty string.

C: total = 30 and in.fail() is true. Extraction fails at x, so the loop ends before the final 30 is processed. Clearing the error alone would leave the offending character to handle.

Sources used for this comparison

Page references in this guide use PDF page numbers, including title slides. All lecture files below are in Midterm/Lecture notes. The source final is in Past papers.

2021 Spring final: 2_CSC3002_Final_2021_Spring.pdf.pdf, pp.2-8. Original questions reproduced in the companion question PDF.

L1: 1-Overview/1-CSC3002_Lecture Notes Overview of C++.pdf. Course outline p.13; foundations pp.16-64.

L2: 2-Functions and Libraries 1/Week 1/2 - CSC3002_Lecture Notes Functions and Libraries Part 1.pdf. Functions, references and operators pp.3-35.

L3: 3 - Function and Library 2/3-CSC3002_Lecture Notes Functions and Libraries Part 2.pdf. Interfaces pp.4-23; random library pp.24-34.

L4: 4 - Strings/4-CSC3002_Lecture Notes_Strings.pdf. C strings pp.10-15; C++ strings pp.22-38.

L5: 5 - Stream/5-CSC3002_Lecture Notes Streams.pdf. Formatting pp.11-17; files pp.20-31; string streams pp.34-38.

L6: 6 - Array/6-CSC3002_Pointers and Arrays(2).pdf. Memory pp.3-19; pointers pp.20-34; arrays pp.35-46.

L7: 7- Dynamic memory management/7-CSC3002 Dynamic memory management.pdf. Allocation pp.5-20; const/static pp.22-26; smart pointers pp.27-30.

L8: 8 - Designing Classes/lecture/8-CSC3002_Designing Classes(1).pdf. Copying pp.21-28; operators pp.31-39; virtual classes pp.41-47; OOP pp.54-57.

L9: 9 - Template and Iterator/9-CSC3002_Template and Iterator.pdf. Templates pp.3-14; iteration pp.16-22.

Existing scope context: output/pdf/CSC3002_Lectures_1-9_Revision_Guide.md, opening scope statement. This is a previously prepared study guide, not an official syllabus notice.

The extracted paper preserves its historical wording and answer choices. The practice guide provides explanations for Q31-32 and the new stream drills; it is not an official marking scheme.